Sorgular (Queries)

0	1	2	3	4	5	6	7
1	3	8	4	6	1	3	4

sum(3,6) = 14

```
int sum(int a, int b) {
    int s = 0;
    for (int i = a; i <= b; i++) {
        s += array[i];
    }
    return s;
}
```

O(n)

Eğer q sorgu yapılacaksa O(nq) zaman gerekir.

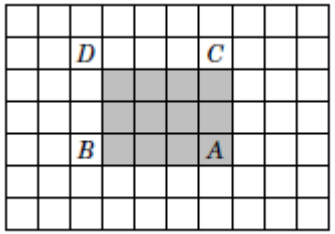
0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

0	1	2	3	4	5	6	7
1	4	8	16	22	23	27	29

$\text{sum}_q(a,b) = \text{sum}_q(0,b) - \text{sum}_q(0,a-1)$

O(1)

sum(3,6) = 19 = 27-8



S(x) = x noktasından sol üst köşeye olan alan olsun

Taralı Alan = S(A)-S(B)-S(C)+S(D)

Sorgular (Queries)

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

Sparse table metodu

Dizideki verilen aralık için minimum sayıyı bulun.

min(3,6)= 1

O(n logn) bir yöntemle tüm sorguları O(1) yapabiliriz.

a	b	$\min_q(a,b)$
0	0	1
1	1	3
2	2	4
3	3	8
4	4	6
5	5	1
6	6	4
7	7	2

a	b	$\min_q(a,b)$
0	1	1
1	2	3
2	3	4
3	4	6
4	5	1
5	6	1
6	7	2

a	b	$\min_q(a,b)$
0	3	1
1	4	3
2	5	1
3	6	1
4	7	1
0	7	1

$b-a+1=2^0$ $b-a+1=2^1$ $b-a+1=2^2$

$$\min_q(a,b) = \min(\min_q(a,a+w-1), \min_q(a+w,b)),$$

$$w = (b-a+1)/2.$$

min(1,6) = min (min(1,3),min(4,6))

$$\min_q(a,b) = \min(\min_q(a,a+k-1), \min_q(b-k+1,b)).$$

k (b-a+1)'i geçmeyen 2'nin üssü -> a=1 & b=6 için k=4

min(2,6) = min (min(2,5),min(3,6)) = 1
min(1,6) = min (min(1,4),min(3,6)) = 1

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

O(1) zamanda tüm minimumları bulabiliriz

Çalışma: k sayısını nasıl buluruz?

Sorgular (Queries)

Binary indexed tree (Fenwick Tree)

$$\mathtt{tree}[k] = \mathtt{sum}_q(k - p(k) + 1, k),$$

p(k) -> k yı bölen en büyük 2'nin kuvveti

p(100) = 4

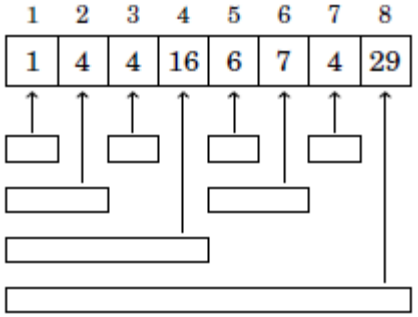
P(13) = 1

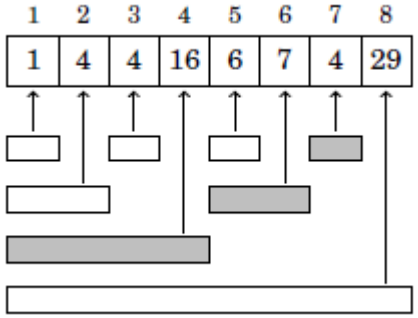
P(16) =16

p(k) = k & -k

1	2	3	4	5	6	7	8
1	3	4	8	6	1	4	2

1	2	3	4	5	6	7	8
1	4	4	16	6	7	4	29





sum(1,k)

```
int sum(int k) {
    int s = 0;
    while (k >= 1) {
        s += tree[k];
        k -= k&-k;
    }
    return s;
}
```

$$\mathtt{sum}_q(a,b) = \mathtt{sum}_q(1,b) - \mathtt{sum}_q(1,a-1).$$

sum(1,7) = sum(1,4)+sum(5,6)+sum(7,7)=27

Orijinal dizide k. sıradaki değer x artarsa ağaç aşağıdaki şekilde güncellenir

```
void add(int k, int x) {
    while (k <= n) {
        tree[k] += x;
        k += k&-k;
    }
}
```

Coin Problem

$\text{coins} = \{1, 3, 4\}$

solve(x) x birim için gerekli min para sayısı

$$
\begin{aligned}
\text{solve}(0) &= 0 \\
\text{solve}(1) &= 1 \\
\text{solve}(2) &= 2 \\
\text{solve}(3) &= 1 \\
\text{solve}(4) &= 1 \\
\text{solve}(5) &= 2 \\
\text{solve}(6) &= 2 \\
\text{solve}(7) &= 2 \\
\text{solve}(8) &= 2 \\
\text{solve}(9) &= 3 \\
\text{solve}(10) &= 3
\end{aligned}
$$

$$
\begin{aligned}
\text{solve}(x) = \min(&\text{solve}(x-1)+1, \\
&\text{solve}(x-3)+1, \\
&\text{solve}(x-4)+1).
\end{aligned}
$$

$$
\text{solve}(x) = \begin{cases} \infty & x < 0 \\ 0 & x = 0 \\ \min_{c \in \text{coins}} \text{solve}(x-c)+1 & x > 0 \end{cases}
$$

```
int solve(int x) {
    if (x < 0) return INF;
    if (x == 0) return 0;
    int best = INF;
    for (auto c : coins) {
        best = min(best, solve(x-c)+1);
    }
    return best;
}
```

memoization

```
int solve(int x) {
    if (x < 0) return INF;
    if (x == 0) return 0;
    if (ready[x]) return value[x];
    int best = INF;
    for (auto c : coins) {
        best = min(best, solve(x-c)+1);
    }
    value[x] = best;
    ready[x] = true;
    return best;
}
```

Coin problem

$\text{coins} = \{1,3,4\}$

solve(x) x birim için çözüm sayısı

5 elde etmek için çözümler-> solve(5)=6

- 1+1+1+1+1
- 1+1+3
- 1+3+1
- 3+1+1
- 1+4
- 4+1

$$\begin{aligned}\text{solve}(x) = {} & \text{solve}(x-1) + \\ & \text{solve}(x-3) + \\ & \text{solve}(x-4).\end{aligned}$$

$$\text{solve}(x) = \begin{cases} 0 & x < 0 \\ 1 & x = 0 \\ \sum_{c \in \text{coins}} \text{solve}(x-c) & x > 0 \end{cases}$$

```
#include <iostream>
#include <bits/stdc++.h>

#define INF 20000;

using namespace std;


int main()
{
 int coins[3]={1,3,4};
 int count1[100]={0};
 count1[0]=1;

 for(int x=1;x<=5;x++){
 for(auto c:coins)
 {
     if(x-c >=0)
     {
         count1[x] += count1[x-c];
     }
 }


 }
cout<<count1[5];
}
```

Yol(path) problemi

3	7	9	2	7
9	8	3	5	5
1	7	9	8	5
3	8	6	4	10
6	3	9	7	8

En üst soldan en alt sağa kadar alta ve sağa giderek oluşan yollar içinde değer toplamı değeri en büyük olan yol hangisidir?
Cevap : 67

$$\text{sum}(y,x) = \max(\text{sum}(y,x-1), \text{sum}(y-1,x)) + \text{value}[y][x]$$

Sum(0,0) = 0

```
#include <iostream>
#include <bits/stdc++.h>


using namespace std;



int main()
{
 int N=5;
 int sum[6][6]={0};
 int value[N][N]={3,7,9,2,7,9,8,3,5,5,1,7,9,8,5,3,8,6,4,10,6,3,9,7,8};
 for (int y=1;y<=N;y++) {
    for (int x=1;x<=N;x++)
 {

  sum[y][x] = max(sum[y][x-1],sum[y-1][x])+value[y-1][x-1];

 }
 }

cout<<sum[5][5];
}
```

Yol(path) problemi

3	7	9	2	7
9	8	3	5	5
1	7	9	8	5
3	8	6	4	10
6	3	9	7	8

En üst soldan en alt sağa kadar alta ve sağa giderek oluşan yollar içinde değer toplamı değeri en büyük olan yol hangisidir?
Cevap : 67

$$\text{sum}(y,x) = \max(\text{sum}(y,x-1), \text{sum}(y-1,x)) + \text{value}[y][x]$$

sum(y,x) = 0 y ve x 0 ise

```
#include <iostream>
#include <bits/stdc++.h>


using namespace std;



int main()
{
 int N=5;
 int sum[6][6]={0};
 int value[N][N]={3,7,9,2,7,9,8,3,5,5,1,7,9,8,5,3,8,6,4,10,6,3,9,7,8};
 for (int y=1;y<=N;y++) {
    for (int x=1;x<=N;x++)
 {

  sum[y][x] = max(sum[y][x-1],sum[y-1][x])+value[y-1][x-1];

 }
 }

cout<<sum[5][5];
}
```

Ödev

1- knapsack problemlerini inceleyin

2- UVA 11235 problemini inceleyin

3- UVA 11450 problemini inceleyin (Bir sonraki ders beraber çözeceğiz)